Optimizing Hash Join Execution in BabyDB:

RPT-Style Bloom Filter Pushdown

Chen Feng

April 2026

Abstract

This report presents an optimization of the hash join operator in BabyDB, a Volcano-model in-memory relational database. We begin by profiling the baseline HashJoinOperator implementation, identifying four concrete performance bottlenecks: per-tuple vector copies, a cache-unfriendly hash table structure, absence of early-rejection filtering, and suboptimal memory allocation patterns. We then implement Robust Predicate Transfer (RPT)-style Bloom filter pushdown [1] to address the early-rejection gap: each HashJoinOperator builds a Bloom filter over its build-side keys and pushes it down the probe pipeline to the originating SeqScanOperator, which applies the filter against raw table rows before any per-tuple extraction occurs. A redesigned, cache-friendly Bloom filter implementation eliminates the per-probe overhead present in an earlier attempt. The result is a mean speedup of 4.9× across all 10 Join Order Benchmark queries (total time: 11,900 ms → 2,431 ms), with per-query improvements ranging from 2.9× to 19.4×. All queries produce correct output cardinality and all existing unit tests pass.

1 Introduction

BabyDB implements multi-table joins using the Volcano (iterator) model. Each operator exposes a Next() interface that pulls one Chunk at a time from its children. The benchmark suite consists of 10 queries derived from the Join Order Benchmark (JOB), each of which is a multi-way hash join tree over pre-loaded in-memory tables. The harness measures end-to-end wall-clock time from plan construction through final output.

The baseline HashJoinOperator materialises the build side into a std::unordered_multimap on the first call to Next(), then probes it chunk-by-chunk as tuples are pulled from the probe child. While functionally correct, this implementation has several performance gaps that become significant at the scale of JOB queries. This report documents those gaps, our optimization strategy, the implementation details, and the resulting performance.

2 Baseline Analysis

We examined the baseline source code and identified four categories of inefficiency. These are summarised in Table 1 and discussed in detail below.

Bottleneck	Root Cause	Cost Impact
Per-tuple copy in Next()	new_tuple = probe_tuple; then insert build tuple → new vector allocation on every output row	O(width) allocation + copy per output tuple; dominates when joins produce many rows
std::unordered_multimap	Separate chaining with pointer-based buckets; one heap allocation per entry	Poor cache locality; high per-entry overhead
No early rejection	Every probe tuple reaches equal_range regardless of whether it matches	On selective joins, many probe tuples produce no output yet pay full hash-table cost

Bottleneck	Root Cause	Cost Impact
Chunk allocation in Next()	output_chunk.reserve() and emplace_back without pre-sizing	Repeated realloc when output_chunk grows beyond capacity estimate

Table 1. Performance bottlenecks in the baseline HashJoinOperator.

2.1 Per-Tuple Copying in Next()

For every matching probe/build tuple pair, the baseline constructs the output tuple by first copying the probe tuple into a new vector, then appending the build tuple:

```
new_tuple = probe_tuple;                          // copy #1: vector copy
new_tuple.reserve(new_tuple.size() + width_);
new_tuple.insert(new_tuple.end(),                 // copy #2: append build slice
    tuples_.begin() + match_ite->second,
    tuples_.begin() + match_ite->second + width_);
```

Each call to emplace_back() followed by this pattern triggers a heap allocation for the new tuple vector. On queries that produce millions of output rows — such as 16a and 17a — this dominates execution time. The allocation itself is O(width) per output tuple, and the resulting working set is fragmented across the heap, destroying cache locality in downstream operators.

2.2 std::unordered_multimap Hash Table

std::unordered_multimap uses separate chaining with individually heap-allocated nodes. Each node carries bookkeeping overhead (next pointer, hash cache, allocator metadata) in addition to the key-value pair. For a build side of N tuples this means N separate allocations during BuildHashTable(), and N pointer dereferences — each likely causing a cache miss — during the probe phase.

An open-addressing hash table with linear probing would pack all entries into a contiguous array, reducing the probe to sequential cache-line reads. For the implementation in this report we retain the unordered_multimap but add the Bloom filter layer to short-circuit most of the probes entirely.

2.3 No Early Rejection of Non-Matching Probe Tuples

The most structurally important gap in the baseline is the absence of any mechanism to reject probe tuples that will find no match in the build side. In a selective join — one where a large fraction of probe-side rows have no matching build-side key — every such tuple must:

- be read from the raw table by SeqScanOperator;
- have its join-key column extracted via KeysFromTuple();
- be placed into a Chunk and passed up through every intermediate operator;
- survive any intermediate hash-join probes (including UnionTuple calls);
- finally reach equal_range() in HashJoinOperator, which returns an empty range and produces no output.

All five steps are paid in full for every non-matching tuple, yet the tuple contributes zero rows to the result. On JOB-style queries that join large movie-metadata tables against small filter tables, the non-match fraction can exceed 90%, meaning more than nine-tenths of all pipeline work is wasted. This is the primary target of our optimization.

2.4 Memory Allocation in Next()

output_chunk is reserved to CHUNK_SUGGEST_SIZE at the start of each Next() call, but when the output grows beyond that estimate the chunk is reallocated with reserve(size * 2). On high-cardinality joins this triggers repeated reallocation, each of which copies the existing contents to a new allocation. Pre-computing an output size bound or reserving conservatively at construction time would eliminate this.

3 Prior Attempt: In-Operator Bloom Filter

Before arriving at the pushdown approach, we implemented a Bloom filter inside HashJoinOperator itself — the natural first intuition. The filter was built over build-side keys in BuildHashTable() and consulted in Next() before each equal_range call:

```
if (bloom_filter_ && !bloom_filter_->might_contain(probe_key)) {
    continue;  // skip equal_range for non-matching keys
}
auto match_range = pointer_table_.equal_range(probe_key);
```

This version ran consistently ~10–14% slower than the baseline (mean: 13,215 ms vs. 11,900 ms). The reason is structural: by the time a probe tuple reaches the BF check inside Next(), all five pipeline steps listed in Section 2.3 have already occurred. The only work the filter could save was the equal_range call itself — the single cheapest step in the pipeline. The filter's own probe cost (involving std::vector<bool> bit-pack overhead and 64-bit modulo indexing up to k=10 times) exceeded the saved cost on most tuples. The correct fix is not a faster filter at this position; it is moving the filter to a position where it can eliminate all upstream work.

4 Optimization: RPT-Style Bloom Filter Pushdown

4.1 Core Idea

The Robust Predicate Transfer paper [1] identifies a simple structural requirement: a filter is only effective when placed before the work it intends to prevent. In a Volcano pipeline, this means placing the filter at the source — the SeqScanOperator — where it can reject raw table rows before any column extraction, chunk insertion, or intermediate-join processing occurs.

The pushdown protocol is as follows. At plan construction time, each HashJoinOperator allocates a BloomFilter and calls RegisterBloomFilter() on its probe child, passing the BF and the probe column name. This call propagates down the probe pipeline through any intermediate operators until it reaches a SeqScanOperator, which stores the BF. During execution, BuildHashTable() inserts all build-side keys into the BF and calls MarkReady(). From that point, every raw table row rejected by the BF in the scan never enters the pipeline at all.

4.2 Redesigned Bloom Filter

The in-operator filter's per-probe cost (Section 3) motivated a full redesign. Table 2 summarises the changes between the initial and final implementations.

Property	Initial BF	RPT BF	Rationale
Backing store	std::vector<bool>	std::vector<uint64_t>	vector<bool> forces bit-packed access with per-element iterator overhead; uint64_t allows direct shift/mask in one instruction
Probe index	(h1 + i·h2) % bits_.size()	(h1 + i·h2) & mask_	64-bit IDIV for modulo (~20–90 cycle latency); replaced by single AND when capacity is a power of two
Hash function	std::hash<data_t> + ad-hoc shift	splitmix64 + independent mix for h2	std::hash on integers is often an identity or weak mix; splitmix64 provides strong avalanche at near-zero cost
k (probes / key)	Clamped at 10	Clamped at 6	k > 6 probes additional cache lines whose latency cost exceeds the marginal reduction in false positives
Readiness guard	Always active from construction	ready_ flag; MightContain returns true until MarkReady()	Without a guard, the filter rejects all tuples before it is populated, violating correctness

Table 2. Design differences between the initial and RPT Bloom filter implementations.

The ready_ flag requires additional explanation. The BF is allocated and pushed to the scan at plan construction time, before any data is read. While ready_ is false, MightContain always returns true (pass-through). MarkReady() is called at the end of BuildHashTable(), after every build-side key has been inserted. Volcano semantics guarantee that the build phase completes before the probe phase begins, so MarkReady() is always called before the scan produces its first tuple. The invariant holds for arbitrary plan depth, as shown in Section 4.5.

4.3 Operator Interface Extension

One virtual method was added to the base Operator class. Its default body is a no-op, which is safe for any operator that cannot forward column provenance:

```
// operator.hpp
virtual void RegisterBloomFilter(
    std::shared_ptr<BloomFilter> bf,
```

```
    const std::string& column_name) {}  // default: no-op
```

Two operators override this method. HashJoinOperator forwards the BF to whichever child contains column_name in its output schema (guaranteed unique by CheckSchema). SeqScanOperator stores the registration as a pending entry, resolved to a raw column index in SelfInit() so the hot path in Next() uses only integer arithmetic.

4.4 Pushdown at Construction Time

In the HashJoinOperator constructor, immediately after the child operators are recorded:

```
bloom_filter_ = std::make_shared<BloomFilter>(1024, 0.01);
probe_child_operator->RegisterBloomFilter(bloom_filter_, probe_column_name_);
```

Because plan construction in BabyDB is bottom-up, by the time the outermost join is constructed, every BF has already been forwarded to its target leaf scan. A placeholder capacity of 1024 is used; in BuildHashTable(), after the actual tuple count is known, the BF is resized by move-assigning a new BloomFilter into the object behind the shared_ptr, so the scan automatically sees the correctly-sized filter:

```
// After materialising all build tuples:
if (tuple_count_ > 1024)
    *bloom_filter_ = BloomFilter(tuple_count_, 0.01);
else
    bloom_filter_->Clear();
for (idx_t i = 0; i < tuple_count_; i++) {
    bloom_filter_->Insert(tuples_[i * width_ + build_key_attr]);
}
bloom_filter_->MarkReady();
```

4.5 Filtering at the Scan

SeqScanOperator::Next applies registered filters before KeysFromTuple:

```
if (has_filters) {
    bool rejected = false;
    for (const auto& rf : resolved_filters_) {
        if (!rf.bf->MightContain(tuple[rf.raw_column_idx])) {
            rejected = true; break;
        }
    }
    if (rejected) continue;
}
// Only here: extract columns, insert into chunk
```

A rejected row pays only the BF probe cost — two hash evaluations and up to six bit-array reads. It never reaches KeysFromTuple, never enters a Chunk, and never triggers any join probe above the scan.

4.6 Correctness of the Timing Invariant

The safety requirement is that every BF held by a scan is ready before that scan produces its first tuple. The following Volcano execution trace proves this for a two-level join; the argument extends to arbitrary depth by induction.

```
outer.Next()
  └─ BuildHashTable()       # outer.bf populated; MarkReady() called
      └─ drains outer build side completely
  └─ probe phase begins
      └─ inner.Next()
          └─ BuildHashTable()   # inner.bf populated; MarkReady() called
              └─ drains inner build side completely
          └─ probe phase begins
              └─ scan.Next()    # both outer.bf and inner.bf are ready_
```

For any leaf scan, every ancestor join that registered a BF on it has fully completed its build phase before that scan produces its first output. The invariant is guaranteed purely by the pull model, with no synchronisation required.

5 Implementation Summary

Table 3 lists all files modified or created. The benchmark harness and all test files are unchanged.

File	Status	Changes
bloom_filter.hpp	**New**	Header-only BloomFilter: uint64_t words, power-of-two mask, splitmix64, k ≤ 6, ready_ flag
operator.hpp	**Modified**	Added virtual RegisterBloomFilter() with default no-op; forward declaration of BloomFilter
hash_join_operator.hpp	**Modified**	bloom_filter_ changed to shared_ptr<BloomFilter>; RegisterBloomFilter override declared; BloomFilter class definition removed (moved to bloom_filter.hpp)
hash_join_operator.cpp	**Modified**	Constructor pushes BF to probe child; BuildHashTable populates BF and calls MarkReady(); in-operator BF check removed from Next()
seq_scan_operator.hpp	**Modified**	Added PendingFilter / ResolvedFilter structs; pending_filters_ and resolved_filters_ members; RegisterBloomFilter override declared
seq_scan_operator.cpp	**Modified**	RegisterBloomFilter stores pending entry; SelfInit resolves column names to raw indices; Next filters before KeysFromTuple

Table 3. Files modified or created.

6 Performance Results

6.1 Overall Comparison

All three versions were run three times on the reduced_data dataset. Times are end-to-end wall-clock milliseconds. All 10 queries produce correct output cardinality.

Version	Run 1 (ms)	Run 2 (ms)	Run 3 (ms)	Mean (ms)	vs. Baseline
Baseline	11,875	11,930	11,896	**11,900**	**1.0×**
Initial BF (no pushdown)	13,110	13,569	12,965	**13,215**	**0.90×**
RPT pushdown (this work)	2,570	2,381	2,341	**2,431**	**4.9×**

Table 4. End-to-end benchmark times across three runs.

The RPT implementation achieves a mean 4.9× speedup over the baseline. The prior in-operator BF attempt is 10% slower than the baseline, confirming the analysis in Section 3: placing a filter after the pipeline cost has been paid is worse than no filter.

6.2 Per-Query Breakdown

Query	Baseline (ms)	RPT (ms)	Speedup
10a	377	38	**9.9×**
11a	457	41	**11.1×**
12a	293	17	**17.2×**
13a	1,654	238	**6.9×**
14a	505	47	**10.7×**
15a	339	18	**18.8×**
16a	3,797	936	**4.1×**
17a	3,402	1,161	**2.9×**
18a	1,222	63	**19.4×**
19a	319	25	**12.8×**

Table 5. Per-query execution time with RPT pushdown versus baseline.

6.3 Analysis

Eight of the ten queries achieve speedups of 6× or greater, indicating that the BF rejects the large majority of probe-side scan rows. Queries 12a, 15a, and 18a show speedups above 17×, corresponding to joins with highly selective build sides where almost no probe row finds a match.

The two outliers, 16a (4.1×) and 17a (2.9×), are the queries with the highest absolute baseline times. Their probe pipelines produce large intermediate result sets, meaning a substantial fraction of scan rows do find matches, reducing the BF rejection rate. Even so, the partial elimination of non-matching rows still yields a meaningful 3–4× improvement. In the limit — a Cartesian join where every probe row matches — the BF rejection rate is zero and the overhead of the BF probe in the scan would appear as a mild regression; this pathological case does not arise in the JOB workload.

7 Trade-offs and Limitations

7.1 Memory Overhead

Each HashJoinOperator allocates one BloomFilter. At a 1% false-positive rate, the optimal bit count is approximately 9.6 bits per inserted key. For the largest build sides in the JOB benchmark (up to a few million rows), this is a few megabytes per join — well within the available memory. The implementation caps the BF at 64 MB to prevent pathological cases.

7.2 False Positives

At a 1% false-positive rate, roughly 1 in 100 non-matching probe rows passes the BF and reaches the hash table. This is correct behaviour (no false negatives; output cardinality is unaffected), and the cost of the spurious equal_range call is negligible compared to the savings from the 99% that are correctly rejected at the scan.

7.3 Re-execution

SelfInit() resets bloom_filter_ via Clear(), which also resets ready_ to false. Pending filter registrations in SeqScanOperator are set at construction time and persist across re-executions; resolved_filters_ is rebuilt at each SelfInit() call on the scan, ensuring correctness if schemas change between executions.

8 Conclusion

The baseline HashJoinOperator has four performance bottlenecks, the most significant being the absence of any early-rejection filter for non-matching probe tuples. An initial attempt to add a Bloom filter inside the operator made things worse, because it intercepted tuples after every upstream pipeline cost had already been paid.

The RPT-style fix moves the filter to its correct position: the SeqScanOperator applies the BF against raw table rows before any extraction, so rejected tuples never enter the pipeline. A redesigned BF using uint64_t backing, power-of-two mask indexing, splitmix64 hashing, k ≤ 6, and a ready_ correctness guard eliminates the per-probe overhead of the initial implementation. The combined result is a 4.9× mean speedup with no correctness regressions.

The central lesson is general: the value of a filter in a pull-based pipeline is proportional to the amount of downstream work it prevents. Only a filter placed at the very bottom of the pipeline — before the first operator — can eliminate the full per-tuple cost.

References

[1] Junyi Zhao, Kai Su, Yifei Yang, Xiangyao Yu, Paraschos Koutris, and Huanchen Zhang. Debunking the Myth of Join Ordering: Toward Robust SQL Analytics. Proc. VLDB Endow., 17(11):3282–3294, 2024.